

# Studios

## Design Document

### Team 24

Aaron Carter

Jai Barath

Kevin Luo

Samuel Glover

Siddharth Sriram

Table of contents

|                                                           |    |
|-----------------------------------------------------------|----|
| Purpose                                                   | 3  |
| - Functional Requirements                                 | 4  |
| - Non-Functional Requirements                             | 6  |
| Design Outline                                            | 8  |
| Design Issues                                             | 10 |
| - Functional Issues                                       | 10 |
| - Non-Functional Issues                                   | 12 |
| Design Details                                            | 14 |
| - Descriptions of Classes and Interaction between Classes | 15 |
| - Sequence Diagrams                                       | 18 |

## Purpose

Procrastination is a common challenge for students, often leading to rushed or incomplete assignments. Many existing study apps, such as Quizlet and Notion, rely heavily on individual planning and self-motivation, which can be ineffective for students who struggle to stay on track.

Studios aims to solve this problem by providing a comprehensive platform that organizes all study tasks in one place. The app combines a to-do list, calendar, and note-taking interface with AI-powered tools to create personalized study plans, break down complex assignments, and generate PracticeQuestions and flashcards.

To combat procrastination, Studios incorporates reminders, visual progress trackers, and social features such as leaderboards. By integrating these elements into a single platform, Studios streamlines the study process, reduces the need for multiple apps, and helps students focus more on learning material effectively.

## Functional Requirements

### 1. User Creation

#### As a user,

- a. I would like to be able to register for a Studious account.
- b. I would like to be able to login to my Studious account.
- c. I would like to be able to reset my password in case I forget it.
- d. I would like to enable two-factor authentication for enhanced account security. (if time allows)
- e. I would like to update my email address and account settings.
- f. I would like to delete my account and associated data.

### 2. Time Management

#### As a user,

- a. I would like to be able to manually create, edit, and mark tasks completed via a to-do list.
- b. I would like to be reminded through email of my tasks and deadlines, and be able to change notification settings (when and how often).
- c. I would like to filter tasks by course, due date, and priority.
- d. I would like to assign priority levels (high, medium, low) to my tasks.
- e. I would like AI to suggest optimal study times based on my deadlines and priority so that I can manage my time efficiently. (if time allows)
- f. I would like to export my calendar to integrate with other calendar apps (Google Calendar, Outlook) so that I can sync my schedule. (if time allows)
- g. I would like to manually add, edit, or remove events to my calendar.
- h. I would like to set recurring tasks for regular assignments or study sessions.
- i. I would like to add subtasks to break down larger assignments.
- j. I would like to attach files or links to specific tasks for quick reference.
- k. I would like to set credit hours or workload estimates for each course.
- l. I would like to receive AI suggestions for task breakdown based on assignment complexity.

### 3. Study Tools

#### As a user,

- a. I would like AI to generate PracticeQuestions based on my notes.
- b. I would like to generate flashcards based on study plans with AI.
- c. I would like to manually create and edit the content of flashcards as needed.
- d. I would like AI to generate/update practice tests based on the mastery of previously generated PracticeQuestions and flashcards. (if time allows)

### 4. Notes

As a user,

- a. I would like to upload lecture videos to generate AI notes.
- b. I would like to upload lecture slides to generate AI notes.
- c. I would like to generate notes from a lecture in real time using speech-to-text AI. (if time allows)
- d. I would like to use voice input to ask questions to AI or write notes. (if time allows)
- e. I would like to organize notes based on courses and topics.
- f. I would like to use text formatting (font, font size, color, spacing, etc) to make notes easier to review.
- g. I would like to tag notes with keywords to be able to search across notes.

## **5. Milestones and Progress Tracking**

As a user,

- a. I would like to see visual progress trackers (e.g, progress bars).
- b. I would like to earn badges or achievements for completing milestones.
- c. I would like to set custom milestones for each course so that I can track specific goals. (if time allows)
- d. I would like to view detailed analytics of my study time by course and week.
- e. I would like to receive weekly summary reports of my productivity.

## **6. Social**

As a user,

- a. I would like to be able to customize my public profile (profile picture, username, etc).
- b. I would like to view the progress of other users using the application through a global leaderboard.
- c. I would like to be able to add friends. (if time allows)
- d. I would like to be able to manage my friends. (if time allows)
- e. I would like to set my profile visibility (public, friends-only, private). (if time allows)
- f. I would like to view my friends' progress via their own public profiles. (if time allows)
- g. I would like to individually message my friends or other users. (if time allows)
- h. I would like to create or join study groups within the app. (if time allows)
- i. I would like to share notes with other users or study groups. (if time allows)

## **7. User Interface**

As a user,

- a. I would like to upload syllabi or course schedules to generate a study plan using AI.

- b. I would like to view my study plans within their own dedicated interfaces that have course-specific to-do lists and calendars.

## Non Functional Requirements

### 1. Performance

#### As a developer,

- a. I would like the system to respond to standard user actions (e.g., viewing notes, updating to-do list) within 500 ms under normal load.
- b. I would like AI-generated lecture summaries and task breakdowns to be produced within 5-10 seconds.
- c. I would like audio transcription to begin processing within 2 seconds of audio upload.

### 2. Scalability

#### As a developer,

- a. I would like the backend to support 100 concurrent users without significant performance reduction.
- b. I would like the database schema to scale horizontally to support the growth of the user base as well as each user's notes, progress, activities, and social network.
- c. I would like the server to be able to handle multiple concurrent users efficiently through the 3-tier client-server architecture.

### 3. Usability

#### As a developer,

- a. I would like the interface to be responsive and usable for both mobile and desktop use.
- b. I would like users to be able to create a note, task, or reminder within a few minutes of logging in.
- c. I would like the app to provide clear visual feedback for each click on a button/page.
- d. I would like the user interface to be well organized, with study plans clearly separated and all components within each plan easily navigable and intuitive to use.

### 4. Security

#### As a developer,

- a. I would like all user authentication to use secure password hashing and JWT-based authentication.

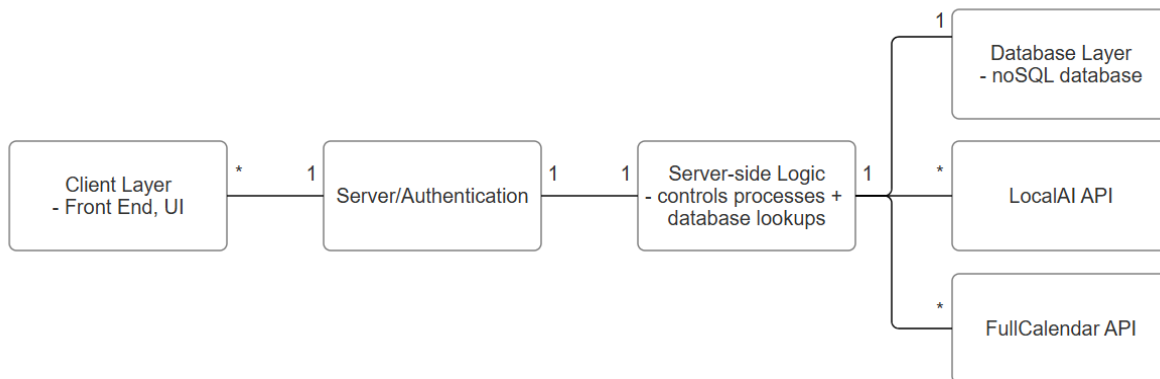
- b. I would like users to only access their own content, notes, and tasks unless explicitly shared.
- c. I would like to use Mongoose to interact with MongoDB to prevent the risk of NoSQL injection.
- d. I would like input validation and sanitization to be applied to all user-provided data to reduce the risk of malicious input.
- e. I would like to use React to protect the system against cross-site scripting by properly encoding user-generated content before rendering it.
- f. I would like the database to be kept behind the server, with the server managing authentication, authorization, and input validation.

# Design Outline

## High Level Overview of System

Our implementation will use a 3-tier Client-Server model. This approach provides a clear separation of concerns: the client handles the user interface, the server manages business logic and AI processing, and the database stores persistent user data. This design improves scalability allowing the server to support multiple concurrent users and social features. Additionally it enhances security by keeping the database behind the server, while the server manages authentication, authorization, and input validation. Centralizing AI tasks on the server ensures efficient processing of features without overloading the client.

Our application will also be a web application ensuring cross platform accessibility so students will be able to use Studious on both desktop and mobile devices. This ensures that students are not restricted by location or by the type of device or operating system they use.



### 1. Client

- The client provides the user interface that users will interact with
- The client will send data to the server through HTTP requests to perform actions such as creating tasks, uploading notes, generating study plans, and integrating with social features
- The client will receive HTTP responses from the server and render the changes in the user interface

### 2. Server

- The server will receive HTTP requests from the client



- b. The server will query the database based on client requests and either add, modify, or retrieve user data
- c. The server will integrate with LocalAI API to process AI requests for task breakdowns, note summarization, and PracticeQuestion generation
- d. The server will integrate with FullCalendar API to sync with users personal calendar
- e. The server will send back a response including the requested data to the client

### **3. Database**

- a. The database will store all persistent data used in Studios including user accounts, to-do lists, notes, study plans, achievements, and social interactions
- b. The database will respond to queries from the server and send the extracted data back
- c. The database ensures data remains available even if the server or client applications are restarted

# Design Issues

## Functional Issues:

### 1. What info do you need to create an account?

- Option 1: email and password only
- Option 2: email, password, phone number
- Option 3: email/password with option social login e.g. Google Account

Choice: Option 3

Justification: Email and password are necessary to identify and authenticate each user securely. Email will also be used for account verification during signup, resetting passwords if forgotten, and delivering task reminders and deadline notifications through emails. A phone number may not be necessary unless alternative security and notifications are sent through SMS. However, it generally costs money to use SMS which we want to avoid. One of our functional requirements if time allows is to integrate tasks with other calendar apps (e.g. Google Calendar) so having the option to create an account using your Google account for example would streamline this. As this is to be done if time allows, we would switch to Option 2 if we are not able to implement this.

### 2. How should we implement the calendar and task management system?

- Option 1: Build custom calendar component from scratch
- Option 2: Use [FullCalendar](#) library

Choice: Option 2

Justification: FullCalendar is a free, open-source (MIT licensed) library that has many features that will give Studious users a great deal of personalization. This includes different calendar views, which we can add settings to toggle amongst. It also supports features like adding events to certain dates and many languages, which are especially useful in an educational setting. There is a [list view](#) feature that can potentially act as our to-do list, where functionality is synced between both e.g. adding an assignment to the calendar will update this list. FullCalendar has full integration with React.

### 3. What approach should we take for the AI note generation from lecture videos?

- Option 1: Process entire video file (audio and visual data), then generate notes
- Option 2: Extract and transcribe audio, then generate notes

Choice: Option 2

Justification: To analyze videos, the AI would have to break down videos into sequences of frames or using a multimodal model (which LocalAI has), but this is extremely resource-intensive, requires more storage, and is much slower than audio transcription. This goes against both our non functional requirements and project objectives. Most

professors post lecture PDFs separately which can be uploaded and processed much faster in this format than they would in a video. Also, the professor talking is the key content of lectures and may include insights that are not shown in the slides displayed on screen.

4. How should we handle the flashcard generation and review system?

- Option 1: AI generates flashcards automatically without user review
- Option 2: AI suggests flashcards that users can approve, edit, or reject
- Option 3: Manual flashcard creation only with optional AI assistance

Choice: Option 2

Justification: We decided on AI suggesting flashcards that allow users to approve, edit, or reject due to user control. Our application is aimed at making it as easy as possible to study and create study materials, so manual flashcard creation provides too much friction between the user and application. Furthermore, we are aiming to allow user customization in order to personalize the process of studying. By having a review system where users can approve, edit, or reject flashcards, this gives users the ability to customize what and how they study.

5. How should we implement the social features and leaderboard?

- Option 1: Public global leaderboard with all users visible
- Option 2: Opt-in leaderboard with privacy controls
- Option 3: Private friend groups only, no global leaderboard

Choice: Option 2

Justification: We decided to have an Opt-in leaderboard with privacy controls due to the ease of implementation compared to private friend groups as well as the privacy allowed. For a global leaderboard, many users might not want their information out there, even if it's as simple as xp tracking. Having an opt-in system allows for more privacy control. In comparison to private friend groups, this would add a lot of implementation questions and details that do not feel core to what we aim for with Studious. Perhaps with enough time, this might be a feature we could add, but it was not something we wanted as a core feature.

## Non-Functional Issues:

### 1. What database should we use?

- Option 1: Use a SQL database with a strict schema
- Option 2: Use a NoSQL database with a flexible schema
- Option 3: Use a hybrid database combining both SQL and NoSQL

#### Choice: Option 2

Justification: We decided on using a NoSQL database, specifically MongoDB. Our application requires that the database horizontally scales with a growing number of users and large volumes of data which MongoDB provides. Studios contains hierarchical data structures due to users having study plans which contain individual components such as to-do list and notes. MongoDB allows data to be stored in flexible nested JSON documents where related information can be stored together.

### 2. What backend framework should we use?

- Option 1: Use Spring Boot (Java)
- Option 2: Use Node.js with Express(Javascript)
- Option 3: Use Django (Python)

#### Choice: Option 2

Justification: We chose Node.js with Express as our backend framework because of its ability to handle many I/O requests concurrently. This will allow it to handle many concurrent client requests at once which is important for our application. Node.js with Express also integrates easily into the MERN stack enabling consistent use of Javascript through the frontend and backend which is a language that we are all familiar with. Having to use another framework in a separate language would introduce additional complexity and a steeper learning curve for this project that we wanted to avoid.

### 3. What model should we use for AI features?

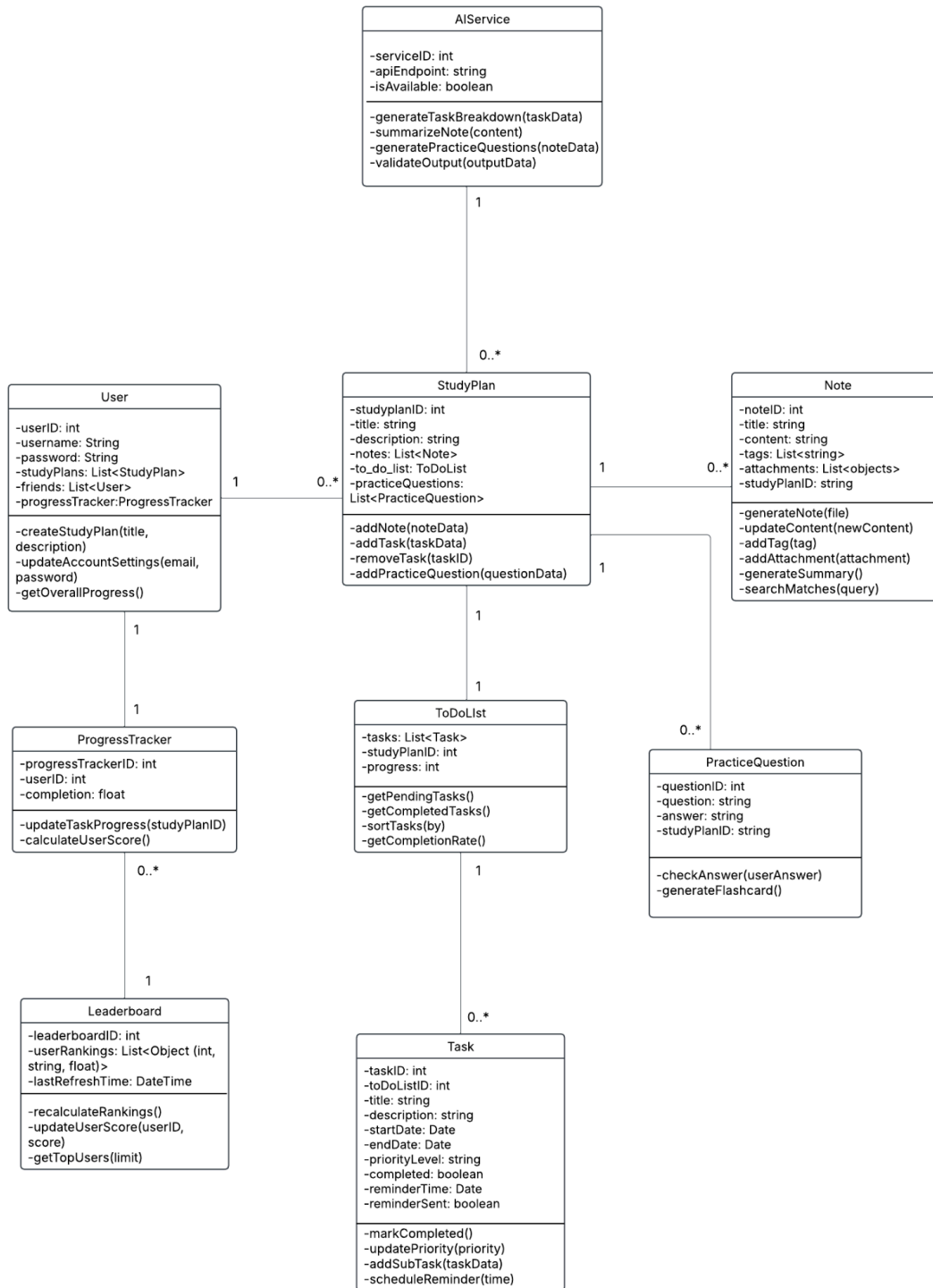
- Option 1: OpenAI API
- Option 2: Google Gemini API
- Option 3: Anthropic Claude API
- Option 4: [LocalAI](#) (open-source) API

#### Choice: Option 4

Justification: There are no pay-per-token fees or subscriptions needed to use LocalAI as it is open-source (MIT licensed), which is ideal since we are trying to avoid costs. It has [audio transcription](#) via Whisper, [image interpretation](#) via LLaVa and GPT Vision, and [text generation](#). These are very important to our planned note-taking features which allow users to upload lecture content (videos and slides) to create notes. Combining audio transcription and image interpretation will allow us to convert videos into notes, and image interpretation can be used to create notes from PDF slides. Being able to generate text will also allow us to implement other study tool features e.g. PracticeQuestions and automating the creation of study sets for flashcards. LocalAI focuses on keeping data local which means that we will be able to store our data within our own NoSQL database, instead of on external servers had we used other cloud-based models. This is subject to change depending on if certain capabilities or the quality of the current ones need to be updated.

# Design Details

## Class Design



## Descriptions of Classes and Interaction between Classes

- **User**

- A user object represents a registered student on the platform
- Each user will have a unique username and password to log in
- Each user will contain a unique user ID identifying them
- Each user will contain an email address where reminders are sent to
- Each user will contain a list of study plan objects that the user owns
- Each user will contain a list of friends that hold the userID's of other users the current user is friends with

- **StudyPlan**

- A study plan object serves as the primary workspace for a user's learning objective containing all individual study components within it
- Each study plan is created when a user creates one and is owned by that user
- Each study plan will contain a unique study plan ID
- Each study plan will contain a title and optional description
- Each study plan will contain a list of notes objects
- Each study plan will contain a to-do list object associated with it
- Each study plan will contain a list of PracticeQuestions and associated answers that can be displayed in test or flashcard format

- **Note**

- A Note object represents study material created or generated for a specific study plan
- Each note will be created by a user and associated with a specific study plan
- Each note can be created manually by user or be auto generated from audio through speech to text
- Each note will contain a unique noteID
- Each note will contain a title
- Each note will contain content representing the main body of the note.
- Each note will contain a list of tags that can be used by users to easily search notes
- Each note will contain a list of attachments that let users keep other study materials within the app directly linked to their notes
- Each note can be processed by the AIService to provide summaries

- **ToDoList**

- A ToDoList object represents a collection of tasks associated with a study plan.
- Each to-do list is associated with a specific study plan
- Each to-do list will contain a list of Task objects
- Each to-do list will track overall completion progress for the study plan based on completion of tasks in list

- **Task**

- A task object represents an individual assignment or study activity within a study plan.
- Each task will contain a unique task id
- Each task is associated with a to-do list
- Each task will contain a title and optional description
- Each task will contain optional start and end dates
- Each task will contain a priority level
- Each task will contain a reminder time and reminder sent attribute for sending reminders to complete the task
- Each task will track will contain a completed attribute which contributes to overall completion progress for the study plan

- **PracticeQuestion**

- A PracticeQuestion object represents a question and answer pair used for studying or assessment.
- Each PracticeQuestion will be associated with a specific StudyPlan
- Each PracticeQuestion will contain a unique question ID.
- Each PracticeQuestion will contain question content and a corresponding answer.
- Each PracticeQuestion can be generated manually or created by the AIService.

- **ProgressTracker**

- A ProgressTracker object monitors the user's study progress.
- Each ProgressTracker will track completion percentages from individual study plans
- Each ProgressTracker will provide data used for visual progress indicators.
- Each ProgressTracker will provide user data on study progress to the global leaderboard

- **Leaderboard**

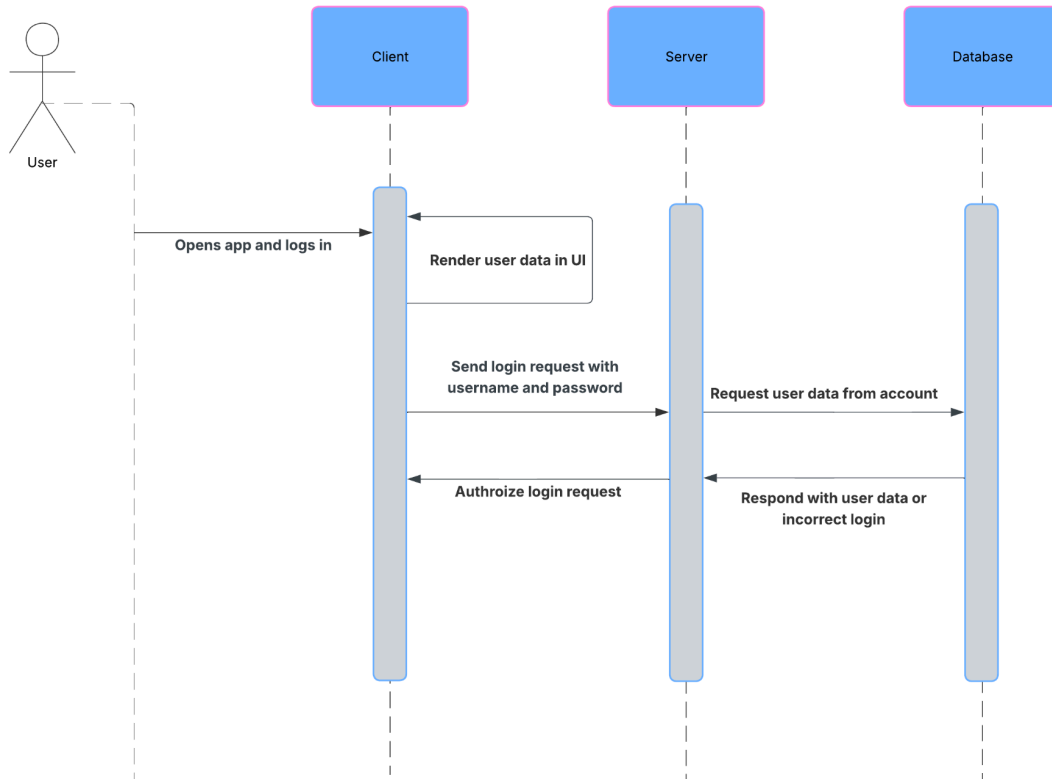
- A Leaderboard object will represent a global ranking system shared by all users on the platform



- The Leaderboard will track and display user rankings based on overall task completion
- The Leaderboard retrieves data from the ProgressTracker.
- The Leaderboard refreshes periodically
- **AIService(LocalAI)**
  - An AIService object will provide an integration with the LocalAI API
  - Each AIService will use LocalAI API to run AI models locally
  - Each AIService will receive data from StudyPlan, Note, and ToDoList objects
  - The AIService can generate summarized versions of notes based on their content
  - The AIService can break down high-level study goals into individual tasks to be added to to-do list
  - The AIService can generate PracticeQuestions and answers based on study plan material
  - The AIService will return structured outputs that are stored within the associated StudyPlan
  - The AIService will not store user data directly and only processes data when requested

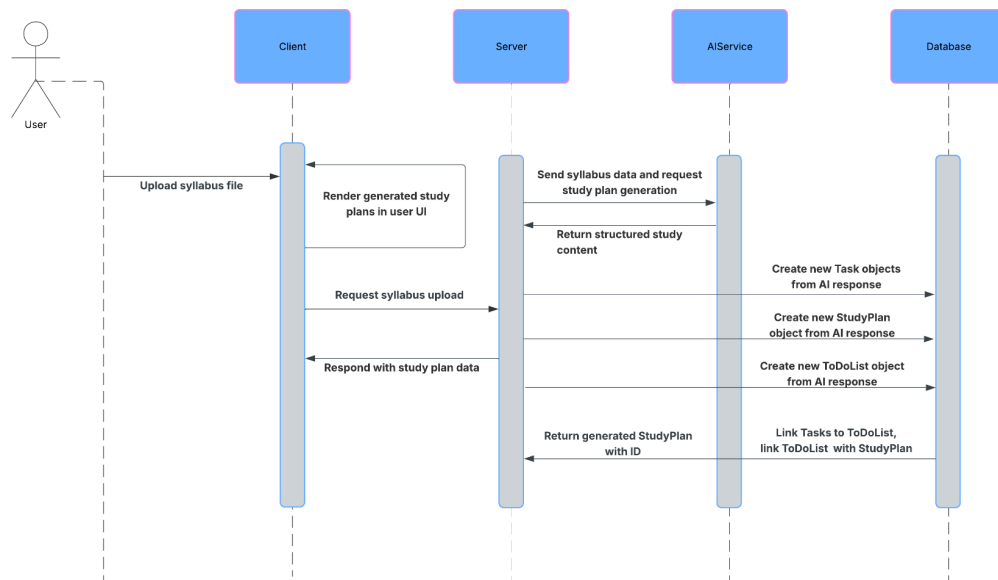
## Sequence Diagrams

Sequence of events when the user starts the program (login)



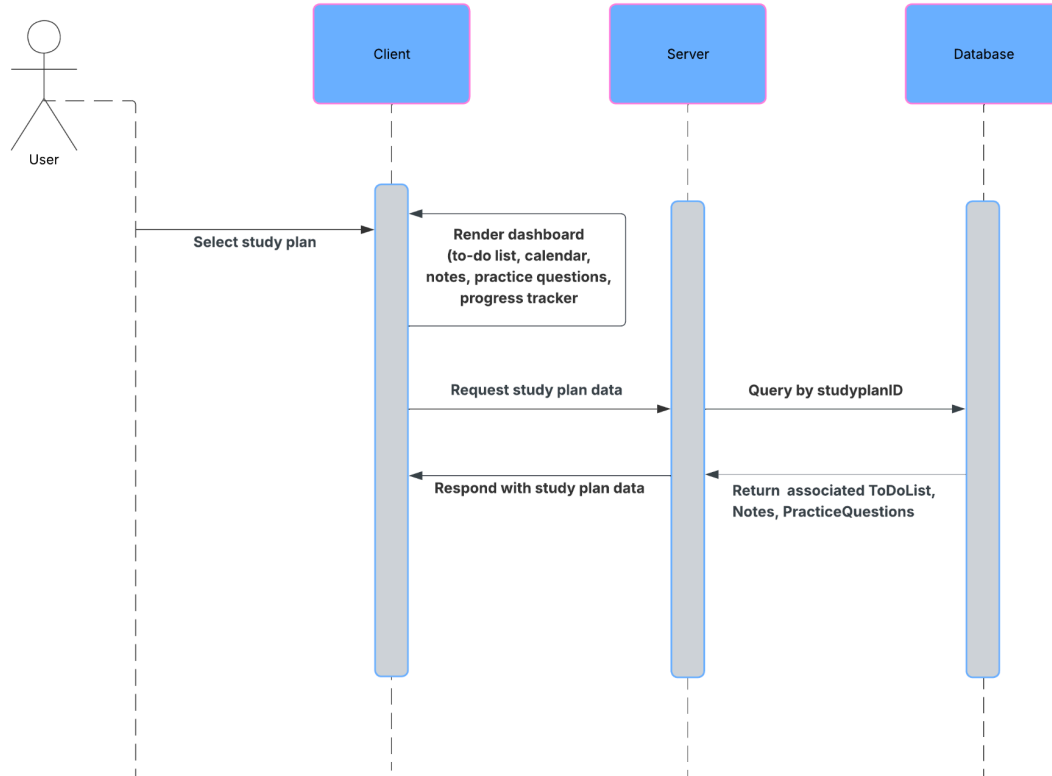
When the user opens up the application, they are prompted a log in screen where they fill out their username and password. These credentials are then sent to the server which queries the database for the user. If the user is authorized then the database sends an authorization back to the server as well as the data associated with the user's account. The data is then rendered on the client UI. If the credentials do not correspond to a valid user account then a response is sent to the client that log in failed and the user is prompted to enter their credentials again.

## Sequence of events when the user generates study from syllabus with AI



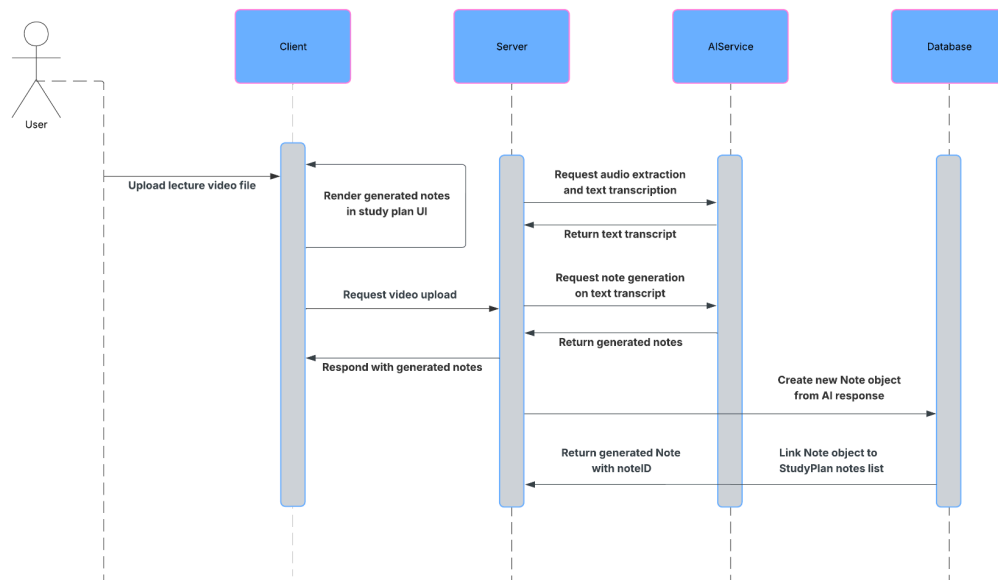
When the user uploads a syllabus file, the client sends a request to the server to process the syllabus. The server forwards the syllabus data to AIService requesting it to generate the new study plan generation. AIService analyzes the syllabus and returns structured study content including the study plan title, description, and a list of tasks with their details. The server then uses this AI-generated data to create objects for the database. It creates Task objects which are linked to the ToDoList object, then the ToDoList is linked to the StudyPlan. Once all objects are created and linked, the database returns the complete StudyPlan with its ID back to the server, which then responds to the client with the study plan data so it can be displayed in the UI.

## Sequence of events when the user enters a study plan dashboard



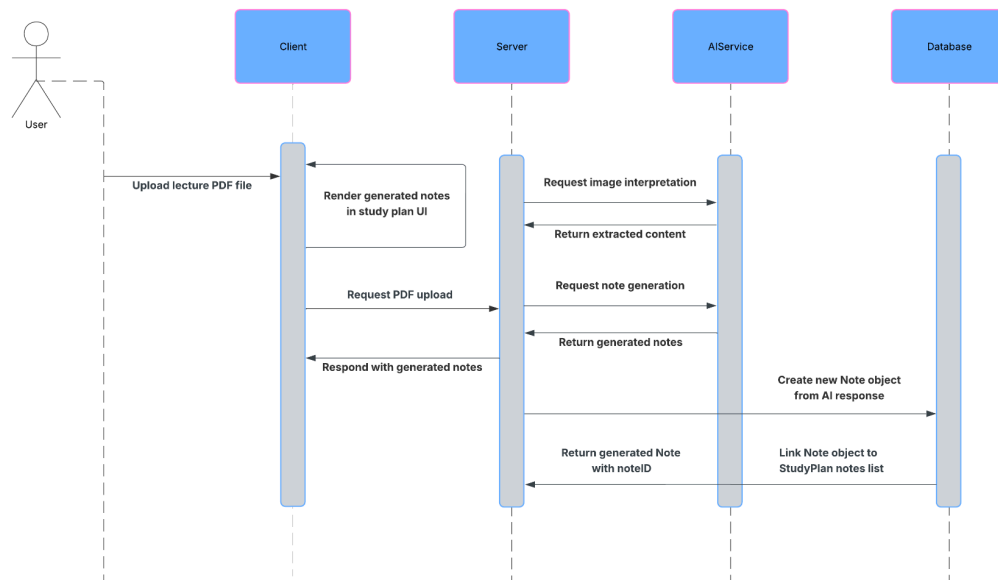
When the user selects a study plan, the client renders the dashboard UI displaying its to-do list, calendar, notes, practice questions, and progress tracker. The client sends a request to the server to retrieve the study plan data. The server queries the database with studyPlanID to fetch the corresponding data. The database returns all associated data including the ToDoList (with all Tasks), Notes, and PracticeQuestions linked to that study plan. The server then responds to the client with the complete study plan data, which then displays the dashboard interface for the user to view and interact with.

## Sequence of events when the user uploads a video file to generate AI notes



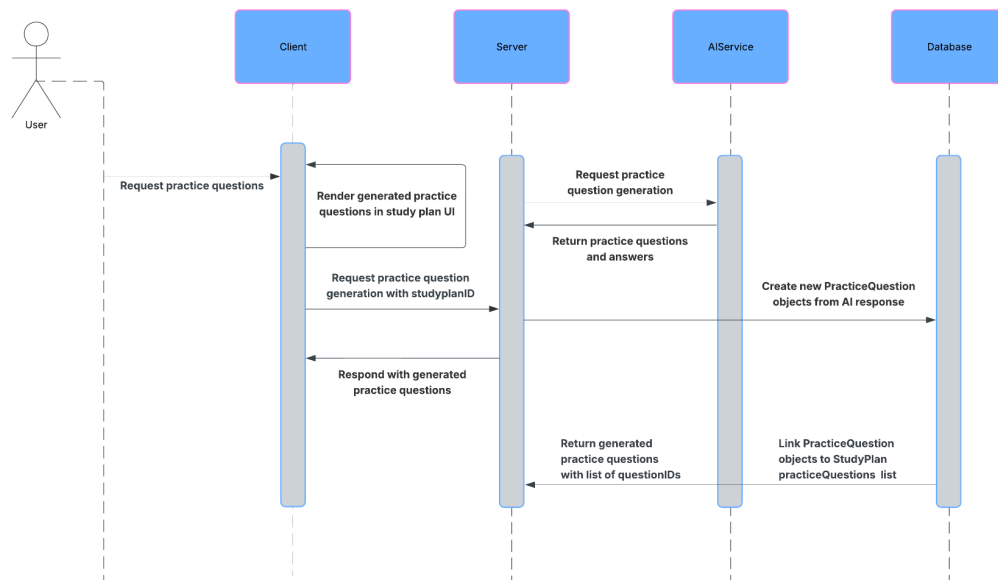
When the user uploads a lecture video file, the client requests the server to process the video. The server forwards the video to AIService, requesting audio extraction and transcription to convert the spoken lecture into text. AIService extracts the audio, transcribes it using speech-to-text, and returns the transcript to the server. The server then sends another request to AIService to generate notes based on the transcript. AIService processes the transcript and returns generated notes. Using this AI-generated content, the server creates a new Note object in the database from the AI response. The database then links the Note object to the StudyPlan's notes list and returns the generated Note with its noteID. The server updates the study plan UI to include the newly generated notes.

## Sequence of events when the user uploads a PDF file to generate AI notes



When the user uploads a lecture PDF file, the client sends a request to the server to process the PDF. The server forwards the PDF to AIService, requesting image interpretation to extract content from the slides. AIService analyzes the PDF and returns the extracted content to the server. The server then sends another request to AIService to generate notes based on the extracted content. AIService processes this information and returns generated notes. Using this AI-generated content, the server creates a new Note object in the database, of which the database links the Note object to the StudyPlan's notes list and returns the noteID. The server updates the study plan UI to include the newly generated notes.

## Sequence of events when the user generates AI practice questions



When the user requests practice question generation, the client sends a request to the server with the studyPlanID. This is done so that the AI knows which study plan to make practice questions on. The server then requests the AIService to generate practice questions, and returns the generated practice questions and answers. Using this AI-generated data, the server creates new PracticeQuestion objects in the database from the AI response. The database then links these PracticeQuestion objects to the StudyPlan's practiceQuestions list and returns a list of the questionIDs. The server updates the study plan UI to include the newly generated notes.